Name Of Course	**Bachelor Of Computer Application (BCA)**
Semester	**4TH**
Name Of Subject	**RDBMS**
Subject Code	**BCA-403**
Marks	**75 Marks**
Lectures	**50 Lectures**

Unit I

1. Introduction and Basic Concepts

1.1 Introduction to RDBMS

1.2 Characteristics of RDBMS

1.3 Applications and Advantages of RDBMS

1.4 Data models – Entity Relationship (ER), Mapping ER Model to Relational Mode, Network.

1.5 Relational and Object Oriented Data Models

1.1 Introduction to RDBMS:

An **RDBMS (Relational Database Management System)** is software used to manage and organize data in a relational database. It stores data in tables consisting of rows and columns, making it easy to access, retrieve, and manipulate using SQL (Structured Query Language).

The core features of RDBMS include:

1. Data Organization:

Data is stored in tables, allowing relationships between different data entities to be defined.

2. ACID Properties:

Ensures reliability and consistency of transactions (Atomicity, Consistency, Isolation, Durability).

3. Data Integrity:

Enforces rules like primary and foreign keys to maintain data accuracy.

4. Scalability:

Supports large datasets and multiple users simultaneously.

5. Popular RDBMS Examples:

MySQL, PostgreSQL, Oracle Database, and Microsoft SQL Server.

RDBMS is widely used for structured data in applications like banking, e-commerce, and healthcare systems.

1.2 Characteristics of RDBMS:

The **Characteristics of RDBMS (Relational Database Management System)** are as follows:

1. Data Storage in Tables:

Data is organized into rows and columns, with each row representing a record and each column representing an attribute.

2. Relationships Between Tables:

RDBMS uses keys (Primary Key and Foreign Key) to establish and enforce relationships between tables.

3. Use of SQL:

SQL (Structured Query Language) is used to query, insert, update, and delete data efficiently.

4. Data Integrity and Accuracy:

Ensures data consistency using constraints like primary keys, foreign keys, and unique constraints.

5. ACID Compliance:

Guarantees reliability during transactions by ensuring Atomicity, Consistency, Isolation, and Durability.

These characteristics make RDBMS a robust choice for managing structured data in various applications.

1.3 Applications and Advantages of RDBMS:

Applications of RDBMS:

1. Banking Systems:

Used to store customer accounts, transaction details, and manage funds securely.

2. E-Commerce:

Manages product catalogs, customer orders, and payment details.

3. Healthcare:

Stores patient records, appointments, and medical histories.

4. Education:

Tracks student data, courses, grades, and faculty information.

5. Government Systems:

Manages citizen records, tax details, and public services efficiently.

Advantages of RDBMS:

1. Data Consistency and Integrity:

Enforces rules to maintain accurate and consistent data using constraints.

2. Ease of Access:

SQL allows easy and flexible querying of data.

3. Data Security:

Provides user authentication and access control to protect sensitive data.

4. Supports Relationships:

Handles complex data relationships through keys and joins.

5. Scalability:

Accommodates growing datasets and multiple concurrent users effectively.

These features make RDBMS essential for modern applications requiring structured and reliable data management.

1.4 Data models – Entity Relationship (ER), Mapping ER Model to Relational Mode, Network:

Data Models in Database Management:

1. Entity-Relationship (ER) Model:

Represents data using **entities** (objects, such as a student or product) and **relationships** (associations between entities, such as enrollment or purchase).

Visualized through an ER Diagram with components like entities (rectangles), relationships (diamonds), and attributes (ovals).

Helps in designing a conceptual schema of the database.

2. Mapping ER Model to Relational Model:

Entities:

Each entity is converted into a table, where attributes become columns.

Relationships:

For **1:1 relationships**, a foreign key is added to one of the related entity tables.

For **1:N relationships**, the primary key of the "one" side becomes a foreign key in the "many" side.

For **M:N relationships**, a separate table is created containing foreign keys from both entities.

Primary Keys:

Ensures unique identification of records in relational tables.

3. Network Data Model:

Represents data as a graph with **nodes** (data entities) and **edges** (relationships).

Supports many-to-many relationships and is more flexible than the hierarchical model.

Uses **pointers** to link records, making data retrieval faster in complex relationships.

Example:

The CODASYL DBTG model.

Summary:

The **ER model** is used for conceptual design, while mapping transforms it into the relational model for implementation.

The **Network model** provides an alternative for representing complex relationships using pointers.

1.5 Relational and Object Oriented Data Models:

Relational Data Model:

The **Relational Data Model** organizes data into **tables (relations)**, consisting of rows (tuples) and columns (attributes).

Key Features:

1. Tabular Structure:

Data is represented in structured tables with clearly defined relationships.

2. Primary and Foreign Keys:

Used to ensure uniqueness and establish relationships between tables.

3. Data Integrity:

Constraints like primary keys, foreign keys, and unique constraints maintain consistency.

4. SQL Support:

SQL is used for data manipulation and querying.

5. ACID Properties:

Ensures reliable and consistent transactions.

Use Cases:

Banking, e-commerce, and other applications where structured data and strict relationships are needed.

Object-Oriented Data Model:

The **Object-Oriented Data Model (OODM)** combines database concepts with object-oriented programming principles.

Key Features:

1. Objects and Classes:

Data is represented as objects, encapsulating both data (attributes) and behavior (methods).

2. Inheritance:

Supports class hierarchies, allowing child classes to inherit properties from parent classes.

3. Encapsulation:

Data and methods are bundled together, improving modularity.

4. Relationships:

Supports complex relationships like aggregation and association.

5. Polymorphism:

Allows the same operation to work differently for different objects.

Use Cases:

CAD systems, multimedia applications, and software that require handling complex, unstructured data.

Comparison:

The **Relational model** is ideal for structured data with clear relationships, while the **Object-Oriented model** excels in managing complex, real-world entities and behaviors.

Unit II

2. SQL Statements & Working with Tables

2.1 Introduction to SQL

2.2. SQL Commands and its types (DDL, DML, DQL, DCL, Transaction Control Commands)

2.3 Data types in SQL

2.4 Creating Tables, Selecting from tables, WHERE Clause

2.5 Selecting from tables, DISTINCT Clause, Column aliasing

2.6 Manipulation Table data

2.7 Altering Table structure

2.8 Data Constraints: Unique, Not Null, Primary Key, Foreign Key, Check, Default Constraint

---

2.1 Introduction to SQL:

SQL (Structured Query Language) is a standard programming language used to manage and manipulate relational databases. It allows users to interact with databases to store, retrieve, update, and delete data.

Key Features of SQL:

1. Data Definition Language (DDL):

Commands like `CREATE`, `ALTER`, and `DROP` are used to define and modify database structures.

2. Data Manipulation Language (DML):

Commands like `INSERT`, `UPDATE`, `DELETE`, and `SELECT` are used to manipulate data within tables.

3. Data Querying:

The `SELECT` statement retrieves data from one or more tables using filters and conditions.

4. Data Control Language (DCL):

Commands like `GRANT` and `REVOKE` are used to control access to data.

5. Transaction Control (TCL):

Commands like `COMMIT`, `ROLLBACK`, and `SAVEPOINT` ensure data consistency during transactions.

Advantages of SQL:

Standardized:

Used universally across various relational database systems like MySQL, PostgreSQL, and Oracle.

Powerful and Flexible:

Supports complex queries for data analysis.

User-Friendly:

Easy to learn and use, even for beginners.

SQL is a foundational tool for database management and is widely used in industries for applications like business analytics, e-commerce, and data science.

2.2. SQL Commands and its types (DDL, DML, DQL, DCL, Transaction Control Commands):

SQL commands are categorized based on their purpose in interacting with a relational database.

The main types are:

1. Data Definition Language (DDL):

Used to define and manage the structure of a database.

Commands:

`CREATE`:

To create a database, table, or other objects.

Example:

`CREATE TABLE Students (ID INT, Name VARCHAR(50));`

`ALTER`:

To modify the structure of an existing table.

Example:

`ALTER TABLE Students ADD Age INT;`

`DROP`:

To delete a database or table.

Example:

`DROP TABLE Students;`

2. Data Manipulation Language (DML):

Used to manipulate data stored in tables.

Commands:

`INSERT`:

To add new data.

Example:

`INSERT INTO Students (ID, Name) VALUES (1, 'Ali');`

`UPDATE`:

To modify existing data.

Example:

`UPDATE Students SET Name = 'Ahmed' WHERE ID = 1;`

`DELETE`:

To remove data.

Example:

`DELETE FROM Students WHERE ID = 1;`

3. Data Query Language (DQL):

Used to query and fetch data from the database.

Command:

`SELECT`:

To retrieve data.

Example:

`SELECT * FROM Students WHERE Age > 18;`

4. Data Control Language (DCL):

Used to control user access and permissions.

Commands:

`GRANT`:

To provide access rights.

Example:

`GRANT SELECT ON Students TO User1;`

`REVOKE`:

To remove access rights.

Example:

`REVOKE SELECT ON Students FROM User1;`

5. Transaction Control Commands (TCL):

Used to manage transactions, ensuring consistency and reliability.

Commands:

`COMMIT`:

Saves changes permanently.

Example:

`COMMIT;`

`ROLLBACK`:

Reverts changes made during a transaction.

Example:

`ROLLBACK;`

`SAVEPOINT`:

Creates a point in a transaction to rollback to later.

Example:

`SAVEPOINT SP1;`

Summary:

DDL manages structure, **DML** manipulates data, **DQL** retrieves data, **DCL** controls access, and **TCL** ensures transaction reliability.

These commands collectively provide complete functionality for managing relational databases.

2.3 Data types in SQL:

SQL data types define the type of data that can be stored in a table's columns. They ensure proper storage, retrieval, and processing of data in a database.

1. Numeric Data Types:

Used for storing numeric values, such as integers and decimals.

INT / INTEGER:

Stores whole numbers. Example: `42`

SMALLINT:

Stores smaller integer values.

BIGINT:

Stores very large integers.

DECIMAL / NUMERIC:

Stores fixed-point numbers with defined precision and scale. Example: `DECIMAL(10, 2)` for values like `1234.56`

FLOAT / REAL:

Stores floating-point numbers for approximate values.

2. Character/String Data Types:

Used for storing text or alphanumeric data.

CHAR(n):

Stores fixed-length strings. Example: `CHAR(5)` for values like `"Ali  "`

VARCHAR(n):

Stores variable-length strings. Example: `VARCHAR(50)` for names.

TEXT:

Stores large amounts of text data.

3. Date and Time Data Types:

Used for storing date and time values.

DATE:

Stores dates in the format `YYYY-MM-DD`. Example: `2024-12-31`

TIME:

Stores time in the format `HH:MM:SS`. Example: `14:30:00`

DATETIME / TIMESTAMP:

Stores both date and time. Example: `2024-12-31 14:30:00`

YEAR:

Stores year values. Example: `2024`

4. Binary Data Types:

Used for storing binary data such as images, files, or multimedia.

BLOB (Binary Large Object):

Stores binary data like images or audio.

VARBINARY(n):

Stores variable-length binary data.

5. Boolean Data Type:

BOOLEAN:

Stores true or false values (1 for TRUE, 0 for FALSE).

6. Other Specialized Data Types:

ENUM:

Stores a predefined list of values. Example: `ENUM('Small', 'Medium', 'Large')`

SET:

Stores a set of predefined values, allowing multiple selections.

Summary:

SQL offers a wide range of data types to handle numeric, text, date/time, and binary data efficiently. Choosing the right data type ensures optimized storage, better performance, and data accuracy.

2.4 Creating Tables, Selecting from tables, WHERE Clause:

1. Creating Tables in SQL:

The `CREATE TABLE` statement is used to define the structure of a table, including its columns and their data types.

Syntax:

```
CREATE TABLE table_name (
    column1 datatype constraints,
    column2 datatype constraints,
);
```

Example:

```
CREATE TABLE Students (
    ID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Age INT,
    Grade CHAR(1)
);
```

This creates a table named `Students` with four columns: `ID`, `Name`, `Age`, and `Grade`.

2. Selecting from Tables in SQL:

The `SELECT` statement is used to retrieve data from a table.

Syntax:

```
SELECT column1, column2, ... FROM table_name;
```

Example:

```
SELECT Name, Age FROM Students;
```

This retrieves the `Name` and `Age` columns for all records in the `Students` table.

To select all columns, use `*`:

```
SELECT * FROM Students;
```

3. WHERE Clause in SQL:

The `WHERE` clause is used to filter records based on specific conditions.

Syntax:

```
SELECT column1, column2, ... FROM table_name WHERE condition;
```

Example:

```
SELECT * FROM Students WHERE Age > 18;
```

This retrieves all columns for students whose `Age` is greater than 18.

Example with multiple conditions:

```
SELECT Name FROM Students WHERE Grade = 'A' AND Age < 20;
```

This retrieves the names of students with a grade of 'A' and age less than 20.

Operators in WHERE Clause:

Comparison Operators:

`=`, `>`, `<`, `>=`, `<=`, `<>` (not equal).

Logical Operators:

`AND`, `OR`, `NOT`.

Pattern Matching:

`LIKE`. Example: `WHERE Name LIKE 'A%'` (names starting with 'A').

Range Checking:

`BETWEEN`. Example: `WHERE Age BETWEEN 18 AND 25`.

Summary:

Creating Tables:

Defines the structure of a database table.

Selecting from Tables:

Retrieves data using the `SELECT` statement.

WHERE Clause:

Filters data based on specified conditions, enabling precise queries.

2.5 Selecting from tables, DISTINCT Clause, Column aliasing:

1. Selecting from Tables in SQL:

The `SELECT` statement is used to retrieve data from one or more tables.

Syntax:

SELECT column1, column2, ... FROM table_name;

Example:

SELECT Name, Age FROM Students;

This retrieves the `Name` and `Age` columns from the `Students` table.

Selecting All Columns:

SELECT * FROM Students;

This retrieves all columns from the table.

2. DISTINCT Clause in SQL:

The `DISTINCT` clause is used to eliminate duplicate rows in the results.

Syntax:

SELECT DISTINCT column1, column2, ... FROM table_name;

Example:

SELECT DISTINCT Grade FROM Students;

This retrieves unique values in the `Grade` column.

Multiple Columns with DISTINCT:

SELECT DISTINCT Age, Grade FROM Students;

This retrieves unique combinations of `Age` and `Grade`.

3. Column Aliasing in SQL:

Column aliasing renames a column in the result set to make it more readable or meaningful.

Syntax:

SELECT column_name AS alias_name FROM table_name;

Example:

SELECT Name AS StudentName, Age AS StudentAge FROM Students;

This displays `Name` as `StudentName` and `Age` as `StudentAge` in the output.

Without AS (Optional):

SELECT Name "Student Name", Age "Student Age" FROM Students;

This produces the same result as above.

Summary:

Selecting from Tables:

Retrieves specific or all columns from a table.

DISTINCT Clause:

Ensures unique results by removing duplicate rows.

Column Aliasing:

Renames columns in the result set for better readability or clarity.

2.6 Manipulation Table data:

Manipulating table data involves **inserting**, **updating**, and **deleting** records within a database table. These operations are performed using **Data Manipulation Language (DML)** commands.

1. Inserting Data into a Table:

The `INSERT` statement adds new records to a table.

Syntax:

INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);

Example:

INSERT INTO Students (ID, Name, Age, Grade) VALUES (1, 'Ali', 20, 'A');

This inserts a record with ID `1`, Name `Ali`, Age `20`, and Grade `A` into the `Students` table.

Inserting into All Columns:

INSERT INTO Students VALUES (2, 'Ahmed', 22, 'B');

2. Updating Data in a Table:

The `UPDATE` statement modifies existing records in a table.

Syntax:

UPDATE table_name SET column1 = value1, column2 = value2, ... WHERE condition;

Example:

UPDATE Students SET Age = 21, Grade = 'A+' WHERE ID = 1;

This updates the `Age` to `21` and `Grade` to `A+` for the record where `ID` is `1`.

Note:

Always use the `WHERE` clause to specify which records to update; otherwise, all records will be modified.

3. Deleting Data from a Table:

The `DELETE` statement removes records from a table.

Syntax:

DELETE FROM table_name WHERE condition;

Example:

DELETE FROM Students WHERE ID = 1;

This deletes the record with `ID = 1` from the `Students` table.

Deleting All Records:

DELETE FROM Students;

This deletes all records but keeps the table structure.

4. Truncating Data from a Table:

The `TRUNCATE` command removes all records from a table but resets identity columns.

Syntax:

TRUNCATE TABLE table_name;

Example:

TRUNCATE TABLE Students;

Summary:

INSERT adds new records.

UPDATE modifies existing records.

DELETE removes specific records based on a condition.

TRUNCATE clears all data while preserving the table structure.

These commands enable dynamic manipulation of data within SQL tables.

2.7 Altering Table structure:

The `ALTER TABLE` statement is used to modify the structure of an existing table. This includes adding, modifying, or deleting columns and constraints.

1. Adding a Column:

You can add a new column to an existing table.

Syntax:

ALTER TABLE table_name ADD column_name datatype constraints;

Example:

ALTER TABLE Students ADD Email VARCHAR(50);

This adds a new column `Email` to the `Students` table.

2. Modifying a Column:

You can change the data type, size, or constraints of an existing column.

Syntax:

ALTER TABLE table_name MODIFY column_name new_datatype constraints;

Example:

ALTER TABLE Students MODIFY Age INT NOT NULL;

This modifies the `Age` column to make it a `NOT NULL` column.

For databases like PostgreSQL:

ALTER TABLE Students ALTER COLUMN Age TYPE BIGINT;

3. Renaming a Column:

You can rename an existing column in the table.

Syntax (varies by database)**:**

ALTER TABLE table_name RENAME COLUMN old_name TO new_name;

Example:

ALTER TABLE Students RENAME COLUMN Email TO ContactEmail;

4. Dropping a Column:

You can remove an existing column from the table.

Syntax:

ALTER TABLE table_name DROP COLUMN column_name;

Example:

ALTER TABLE Students DROP COLUMN Grade;

This deletes the `Grade` column from the `Students` table.

5. Adding or Dropping Constraints:

You can add or remove constraints (like primary key, foreign key, or unique constraints).

Adding a Constraint:

ALTER TABLE Students ADD CONSTRAINT pk_students PRIMARY KEY (ID);

Dropping a Constraint:

ALTER TABLE Students DROP CONSTRAINT pk_students;

Summary:

The `ALTER TABLE` statement is a powerful tool for modifying table structures dynamically. It allows you to:

Add new columns.

Modify existing columns.

Rename columns.

Drop columns or constraints.

These features help maintain the database structure as requirements evolve.

2.8 Data Constraints: Unique, Not Null, Primary Key, Foreign Key, Check, Default Constraint:

Data constraints are rules applied to table columns to ensure the validity, accuracy, and integrity of the data. Below are the most commonly used constraints:

1. UNIQUE Constraint:

Ensures that all values in a column are unique (no duplicates).

A table can have multiple `UNIQUE` constraints.

Syntax:

```
CREATE TABLE Students (
    ID INT UNIQUE,
```

```
        Name VARCHAR(50)
    );
```

Example:

```
    ALTER TABLE Students ADD CONSTRAINT Unique_Email UNIQUE (Email);
```

2. NOT NULL Constraint:

Ensures that a column cannot have `NULL` values (i.e., the field must always have a value).

Syntax:

```
    CREATE TABLE Students (
        ID INT NOT NULL,
        Name VARCHAR(50) NOT NULL
    );
```

Example:

```
    ALTER TABLE Students MODIFY Age INT NOT NULL;
```

3. PRIMARY KEY Constraint:

Combines `UNIQUE` and `NOT NULL`. Ensures each row has a unique identifier.
A table can only have one `PRIMARY KEY`.

Syntax:

```
    CREATE TABLE Students (
        ID INT PRIMARY KEY,
        Name VARCHAR(50)
    );
```

Example:

```
    ALTER TABLE Students ADD CONSTRAINT PK_Students PRIMARY KEY (ID);
```

4. FOREIGN KEY Constraint:

Establishes a relationship between two tables by referencing the `PRIMARY KEY` of another table.
Ensures referential integrity.

Syntax:

```
    CREATE TABLE Enrollments (
        StudentID INT,
        CourseID INT,
        FOREIGN KEY (StudentID) REFERENCES Students(ID)
    );
```

Example:

```
    ALTER TABLE Enrollments ADD CONSTRAINT FK_Student FOREIGN KEY (StudentID)
  REFERENCES Students(ID);
```

5. CHECK Constraint:

Ensures that all values in a column meet a specific condition.

Syntax:

```
CREATE TABLE Students (
    ID INT,
    Age INT CHECK (Age >= 18)
);
```

Example:

```
ALTER TABLE Students ADD CONSTRAINT Check_Age CHECK (Age BETWEEN 18 AND 25);
```

6. DEFAULT Constraint:

Assigns a default value to a column when no value is provided.

Syntax:

```
CREATE TABLE Students (
    ID INT,
    Grade CHAR(1) DEFAULT 'C'
);
```

Example:

```
ALTER TABLE Students ADD CONSTRAINT Default_Grade DEFAULT 'C' FOR Grade;
```

Summary:

UNIQUE:

No duplicates in a column.

NOT NULL:

Column must have a value.

PRIMARY KEY:

Uniquely identifies each row.

FOREIGN KEY:

Links two tables.

CHECK:

Validates data against a condition.

DEFAULT:

Assigns a default value when no input is given.

These constraints ensure data integrity and help maintain consistent and accurate databases.

Unit III

3. Operators & SQL Functions & Views

3.1 Arithmetic Operators, Relational Operators

3.2 Comparison Operators BETWEEN, IN, LIKE, IS NULL

3.3 LOGICAL Operators: AND OR NOT

3.4 SQL Functions: Single, Multiple Row Functions

3.5 Single Row Character, Single Row Number, Single Row Date, Single Row Conversion, Single Row General Functions

3.6 Multiple Row Functions

3.7 Views

3.1 Arithmetic Operators, Relational Operators:

Operators are used to perform operations on data. Below is an explanation of **Arithmetic** and **Relational** operators in SQL:

1. Arithmetic Operators:

Used to perform mathematical calculations on numeric data.

Operator	Description	Example	Result
`+`	Addition	`SELECT 5 + 3;`	`8`
`-`	Subtraction	`SELECT 10 - 4;`	`6`
`*`	Multiplication	`SELECT 6 * 2;`	`12`
`/`	Division	`SELECT 15 / 3;`	`5`
`%`	Modulus (remainder of division)	`SELECT 10 % 3;`	`1`

Example with Tables:

SELECT Salary, Salary + 500 AS NewSalary FROM Employees;

This adds 500 to the `Salary` column and displays the result as `NewSalary`.

2. Relational Operators:

Used to compare values between two expressions or columns, returning a boolean (`TRUE` or `FALSE`) result.

Operator	Description	Example	Result
`=`	Equal to	`SELECT * FROM Students WHERE Age = 18;`	Rows where `Age` is `18`.
`!=` or `<>`	Not equal to	`SELECT * FROM Students WHERE Grade != 'A';`	Rows where `Grade` is not `A`.
`>`	Greater than	`SELECT * FROM Products WHERE Price > 100;`	Rows where `Price` is greater than `100`.

`<`	Less than	`SELECT * FROM Products WHERE Price < 50;`	Rows where `Price` is less than `50`.
`>=`	Greater than or equal to	`SELECT * FROM Orders WHERE Quantity >= 10;`	Rows where `Quantity` is `10` or more.
`<=`	Less than or equal to	`SELECT * FROM Orders WHERE Quantity <= 5;`	Rows where `Quantity` is `5` or less.

Combining Arithmetic and Relational Operators:

Arithmetic and relational operators can be combined in queries to perform calculations and apply conditions.

Example:

SELECT Name, Salary FROM Employees WHERE Salary + 500 > 5000;

This retrieves employees whose adjusted salary exceeds 5000.

Summary:

Arithmetic Operators:

Perform mathematical calculations like addition, subtraction, multiplication, etc.

Relational Operators:

Compare values to filter data based on conditions.

These operators enhance the flexibility and functionality of SQL queries for data retrieval and processing.

3.2 Comparison Operators BETWEEN, IN, LIKE, IS NULL:

Comparison Operators in SQL:

SQL comparison operators are used to filter data based on conditions. Here's an explanation of the operators **BETWEEN**, **IN**, **LIKE**, and **IS NULL**:

1. BETWEEN Operator:

Filters data within a specified range.

The range includes the lower and upper bounds.

Syntax:

SELECT column_name FROM table_name WHERE column_name BETWEEN value1 AND value2;

Example:

SELECT * FROM Students WHERE Age BETWEEN 18 AND 25;

This retrieves all students whose age is between 18 and 25, inclusive.

2. IN Operator:

Filters data that matches any value in a given list.
Acts as a shorthand for multiple `OR` conditions.

Syntax:

SELECT column_name FROM table_name WHERE column_name IN (value1, value2, ...);

Example:

SELECT * FROM Students WHERE Grade IN ('A', 'B', 'C');

This retrieves all students with grades 'A', 'B', or 'C'.

3. LIKE Operator:

Filters data based on a pattern using wildcard characters:

`%` matches zero or more characters.

`_` matches a single character.

Syntax:

SELECT column_name FROM table_name WHERE column_name LIKE 'pattern';

Examples:

Retrieve names starting with 'A':

SELECT * FROM Students WHERE Name LIKE 'A%';

Retrieve names ending with 'n':

SELECT * FROM Students WHERE Name LIKE '%n';

Retrieve names containing 'an':

SELECT * FROM Students WHERE Name LIKE '%an%';

4. IS NULL Operator:

Filters records where the column has a `NULL` value.

Syntax:

SELECT column_name FROM table_name WHERE column_name IS NULL;

Example:

SELECT * FROM Students WHERE Email IS NULL;

This retrieves all students who do not have an email address.

For NOT NULL:

SELECT * FROM Students WHERE Email IS NOT NULL;

Summary:

BETWEEN:

Filters data within a range.

IN:

Filters data that matches any value in a list.

LIKE:

Filters data based on a pattern using wildcards.

IS NULL:

Filters records with `NULL` values.

These operators make queries more flexible and powerful by enabling dynamic and specific data filtering.

3.3 LOGICAL Operators: AND OR NOT:

Logical Operators in SQL:

Logical operators are used in the `WHERE` clause to combine multiple conditions and filter data based on logical relationships. The main logical operators are **AND**, **OR**, and **NOT**.

1. AND Operator:

Combines two or more conditions.
Returns `TRUE` if **all conditions** are true.

Syntax:

SELECT column_name FROM table_name WHERE condition1 AND condition2;

Example:

SELECT * FROM Students WHERE Age > 18 AND Grade = 'A';

This retrieves students who are older than 18 **and** have a grade of 'A'.

2. OR Operator:

Combines two or more conditions.
Returns `TRUE` if **at least one condition** is true.

Syntax:

SELECT column_name FROM table_name WHERE condition1 OR condition2;

Example:

SELECT * FROM Students WHERE Grade = 'A' OR Grade = 'B';

This retrieves students who have a grade of either 'A' **or** 'B'.

3. NOT Operator:

Negates a condition.
Returns `TRUE` if the condition is **FALSE**.

Syntax:

SELECT column_name FROM table_name WHERE NOT condition;

Example:

SELECT * FROM Students WHERE NOT Grade = 'A';

This retrieves all students except those with a grade of 'A'.

Combining Logical Operators:

Logical operators can be combined to create complex conditions. Parentheses `()` can be used to control the order of evaluation.

Example:

SELECT * FROM Students
WHERE (Age > 18 AND Grade = 'A') OR (Grade = 'B' AND NOT Age < 20);

This retrieves students who:

1. Are older than 18 and have a grade of 'A', **OR**
2. Have a grade of 'B' but are not younger than 20.

Summary:

AND:

Returns `TRUE` when all conditions are true.

OR:

Returns `TRUE` when at least one condition is true.

NOT:

Reverses the result of a condition.

These operators allow SQL queries to handle complex filtering and make data retrieval more dynamic and precise.

3.4 SQL Functions: Single, Multiple Row Functions:

SQL functions are used to perform operations on data. These can be classified into **Single-Row Functions** and **Multiple-Row (Aggregate) Functions**.

1. Single-Row Functions:

Operate on each row individually.
Return one result per row.
Types: **String**, **Numeric**, **Date**, and **Conversion** functions.

Examples:

a. String Functions:

`UPPER()`:

Converts text to uppercase.

SELECT UPPER(Name) FROM Students;

`LOWER()`:

Converts text to lowercase.

SELECT LOWER(Name) FROM Students;

`CONCAT()`:

Combines two strings.

SELECT CONCAT(FirstName, ' ', LastName) AS FullName FROM Students;

b. Numeric Functions:

`ROUND()`:

Rounds a number to the specified decimal places.

SELECT ROUND(Salary, 2) FROM Employees;

`ABS()`:

Returns the absolute value of a number.

SELECT ABS(-10) AS AbsoluteValue;

c. Date Functions:

`NOW()`:

Returns the current date and time.

SELECT NOW();

`DATEDIFF()`:

Finds the difference between two dates.

SELECT DATEDIFF('2025-01-01', '2024-12-01') AS DaysDifference;

d. Conversion Functions:

`CAST()`:

Converts data from one type to another.

SELECT CAST(123.45 AS INT);

`CONVERT()`:

Converts data types (varies by SQL dialect).

SELECT CONVERT(INT, Salary) FROM Employees;

2. Multiple-Row (Aggregate) Functions:

Perform operations on a group of rows and return a single result.

Commonly used with `GROUP BY`.

Examples:

a. COUNT():

Returns the number of rows.

SELECT COUNT(*) AS TotalStudents FROM Students;

b. SUM():

Returns the total sum of a column.

SELECT SUM(Salary) AS TotalSalary FROM Employees;

c. AVG():

Returns the average value of a column.

SELECT AVG(Age) AS AverageAge FROM Students;

d. MAX():

Returns the highest value in a column.

SELECT MAX(Salary) AS HighestSalary FROM Employees;

e. MIN():

Returns the lowest value in a column.

SELECT MIN(Age) AS YoungestStudent FROM Students;

Key Differences:

Feature	Single-Row Functions	Multiple-Row Functions
Operation Scope	Each row individually	A group of rows
Result	One result per row	One result for multiple rows
Common Examples	`UPPER()`, `ROUND()`, `NOW()`	`SUM()`, `AVG()`, `COUNT()`

Summary:

Single-Row Functions:

Work on each row and return one result per row (e.g., `UPPER()`, `ROUND()`).

Multiple-Row Functions:

Perform calculations on groups of rows and return a single value (e.g., `SUM()`, `COUNT()`).

These functions enhance the flexibility and capability of SQL queries for data manipulation and analysis.

3.5 Single Row Character, Single Row Number, Single Row Date, Single Row Conversion, Single Row General Functions:

Single-Row Functions in SQL:

Single-row functions operate on each row of a table individually and return one result per row. These functions are classified into **Character**, **Number**, **Date**, **Conversion**, and **General Functions**.

1. Single-Row Character Functions:

Character functions operate on text data.

Function	Description	Example	Result
`UPPER()`	Converts text to uppercase	`SELECT UPPER('hello');`	`HELLO`

`LOWER()`	Converts text to lowercase	`SELECT LOWER('HELLO');`	`hello`
`LENGTH()`	Returns the length of a string	`SELECT LENGTH('Hello');`	`5`
`SUBSTR()`	Extracts a substring from a string	`SELECT SUBSTR('Hello', 2, 3);`	`ell`
`CONCAT()`	Concatenates two strings	`SELECT CONCAT('SQL', 'fUNCTIONS');`	`SQL Functions`
`TRIM()`	Removes leading/trailing spaces	`SELECT TRIM('Hello');`	`Hello`

2. Single-Row Number Functions:

Number functions perform mathematical operations.

Function	Description	Example	Result
`ROUND()`	Rounds a number to specified decimals	`SELECT ROUND(123.456, 2);`	`123.46`
`ABS()`	Returns the absolute value	`SELECT ABS(-10);`	`10`
`MOD()`	Returns the remainder of division	`SELECT MOD(10, 3);`	`1`
`POWER()`	Raises a number to a power	`SELECT POWER(2, 3);`	`8`
`CEIL()`	Rounds up to the nearest integer	`SELECT CEIL(4.2);`	`5`
`FLOOR()`	Rounds down to the nearest integer	`SELECT FLOOR(4.7);`	`4`

3. Single-Row Date Functions:

Date functions operate on date and time values.

Function	Description	Example	Result
`SYSDATE()`	Returns the current date and time	`SELECT SYSDATE();`	Current date/time
`DATEADD()`	Adds a specified interval to a date	`SELECT DATEADD('2023-01-01', INTERVAL 7 DAY);`	`2023-01-08`
`DATEDIFF()`	Finds the difference between two dates	`SELECT DATEDIFF('2023-01-10', '2023-01-01');`	`9`
`EXTRACT()`	Extracts part of a date (year, month, etc.)	`SELECT EXTRACT(YEAR FROM '2025-01-01');`	`2025`
`NOW()`	Returns the current timestamp	`SELECT NOW();`	Current timestamp

4. Single-Row Conversion Functions:

Conversion functions change data from one type to another.

Function	Description	Example	Result
`CAST()`	Converts data type	`SELECT CAST('123.45' AS INT);`	`123`
`CONVERT()`	Converts data type (varies by SQL dialect)	`SELECT CONVERT(INT, '456');`	`456`
`TO_CHAR()`	Converts date or number to a string	`SELECT TO_CHAR(SYSDATE(), 'YYYY-MM-DD');`	`2025-01-01`

5. Single-Row General Functions:

General-purpose functions that don't fall into specific categories.

Function	Description	Example	Result
`NVL()`	Replaces `NULL` with a specified value	`SELECT NVL(NULL, 'No Data');`	`No Data`
`COALESCE()`	Returns the first non-NULL value	`SELECT COALESCE(NULL, NULL, 'Default');`	`Default`
`USER()`	Returns the current database user	`SELECT USER();`	Current user name

Summary:

Character Functions:

Operate on strings (e.g., `UPPER`, `CONCAT`).

Number Functions:

Perform mathematical calculations (e.g., `ROUND`, `ABS`).

Date Functions:

Handle date and time data (e.g., `NOW`, `DATEDIFF`).

Conversion Functions:

Convert data types (e.g., `CAST`, `TO_CHAR`).

General Functions:

Handle special cases (e.g., `NVL`, `COALESCE`).

These functions enhance SQL queries, making them powerful and versatile for data manipulation.

3.6 Multiple Row Functions:

Multiple-row functions, also known as **aggregate functions**, operate on a group of rows and return a single result. These functions are used to perform calculations on a set of values, such as summing or averaging data.

Common Multiple-Row Functions:

Function	Description	Example	Result
`COUNT()`	Counts the number of rows	`SELECT COUNT(*) FROM Students;`	Total number of rows
`SUM()`	Returns the sum of numeric values	`SELECT SUM(Salary) FROM Employees;`	Total salary
`AVG()`	Returns the average of numeric values	`SELECT AVG(Age) FROM Students;`	Average age
`MAX()`	Returns the maximum value in a column	`SELECT MAX(Salary) FROM Employees;`	Highest salary
`MIN()`	Returns the minimum value in a column	`SELECT MIN(Age) FROM Students;`	Youngest student's age

Details and Examples:

1. COUNT():

Counts the number of rows in a table or matching a condition.

Example:

SELECT COUNT(*) AS TotalStudents FROM Students;

This returns the total number of students in the table.

2. SUM():

Adds up all the numeric values in a column.

Example:

SELECT SUM(Salary) AS TotalSalary FROM Employees;

This calculates the total salary of all employees.

3. AVG():

Calculates the average value of a numeric column.

Example:

SELECT AVG(Age) AS AverageAge FROM Students;

This returns the average age of students.

4. MAX():

Finds the maximum value in a column.

Example:

SELECT MAX(Salary) AS HighestSalary FROM Employees;

This retrieves the highest salary in the table.

5. MIN():

Finds the minimum value in a column.

Example:

SELECT MIN(Age) AS YoungestStudent FROM Students;

This retrieves the youngest student's age.

Using Aggregate Functions with GROUP BY:

Multiple-row functions are often used with the `GROUP BY` clause to group data before applying the function.

Example:

```
SELECT Department, AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department;
```

This calculates the average salary for each department.

Key Features of Multiple-Row Functions:

1. Work on groups of rows and return a single value.
2. Commonly used in data analysis for summarizing data.
3. Can be combined with `GROUP BY` to group data by a specific column.
4. Can include conditions using the `HAVING` clause for further filtering.

Summary:

Function	Purpose
`COUNT()`	Counts rows.
`SUM()`	Adds numeric values.
`AVG()`	Calculates averages.
`MAX()`	Finds the highest value.
`MIN()`	Finds the lowest value.

Multiple-row functions are powerful tools for summarizing and analyzing large datasets in SQL queries.

3.7 Views:

A **view** in SQL is a virtual table that is based on the result of a `SELECT` query. It does not store data itself but displays data stored in the underlying database tables.

Key Features of Views:

1. Virtual Table:

Acts like a table but does not store data physically.

2. Dynamic:

Always displays the most recent data from the underlying tables.

3. Read-Only or Updatable:

Views can be read-only or updatable, depending on the database system and query complexity.

4. Simplify Queries:

Encapsulate complex SQL queries, making them reusable and easier to understand.

Syntax to Create a View:

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example of a View:

Create a View:

Suppose you want a view to display only employees from the "IT" department.

```
CREATE VIEW IT_Employees AS
SELECT EmployeeID, Name, Salary
```

```
FROM Employees
WHERE Department = 'IT';
```

Query the View:

```
SELECT * FROM IT_Employees;
```

This displays all employees from the "IT" department.

Advantages of Views:

1. Data Abstraction:

Simplifies data access by hiding the complexity of queries.

2. Security:

Restricts user access to specific rows or columns without exposing the entire table.

3. Reusability:

Once created, a view can be reused in multiple queries.

4. Simplifies Maintenance:

Changes in the base table do not require updating the view.

5. Encapsulation:

Complex joins or calculations can be encapsulated into a view.

Updating Data Using Views:

Some views are **updatable**, allowing `INSERT`, `UPDATE`, or `DELETE` operations.

A view is typically updatable if:

It is based on a single table.

It does not use aggregate functions or `DISTINCT`.

Example:

```
UPDATE IT_Employees
SET Salary = Salary + 5000
WHERE EmployeeID = 101;
```

Dropping a View:

To delete a view:

```
DROP VIEW view_name;
```

Summary:

Definition:

A view is a virtual table based on the result of a query.

Usage:

Simplifies complex queries, enhances security, and provides data abstraction.

Example:

A view showing only "IT" department employees.

Syntax:

Use `CREATE VIEW`, `SELECT`, and optionally `WHERE` for filtering.

Views are powerful tools in SQL for organizing, abstracting, and securing data.

Unit IV

4. Sorting, Grouping, Joining and PL/ SQL

4.1 Introduction to Sorting

4.2 Order by Clause, Group by Clause

4.3 Join, Types of Join: Equi Joins, Non Equi Join, Outer Join: Left, Right, Full, Self Join, Cross Join, Joining three tables

4.4 Subqueries & its types

4.5 Overview of PL / SQL

4.5.1 Declarations Section

4.5.2 Executable Commands Section

4.5.3 Exception Handling Section

4.1 Introduction to Sorting:

Sorting in SQL is the process of arranging data in a specific order, either ascending or descending, based on one or more columns. It is performed using the **ORDER BY** clause in a SQL query.

Key Features of Sorting:

1. ORDER BY Clause:

Used to specify the column(s) and order (ascending/descending) for sorting.

2. Default Order:

SQL sorts data in **ascending order** by default.

3. Multiple Columns:

You can sort data by multiple columns, providing a secondary sort order if needed.

4. Custom Sorting:

Allows for case-insensitive or specific data order based on column values.

Syntax:

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column_name [ASC|DESC];
```

ASC:

Sorts in ascending order (default).

DESC:

Sorts in descending order.

Examples:

1. Sorting by a Single Column (Ascending):

```
SELECT Name, Age
FROM Students
```

```
ORDER BY Age;
```

This sorts the data by the **Age** column in ascending order.

2. Sorting by a Single Column (Descending):

```
SELECT Name, Age
FROM Students
ORDER BY Age DESC;
```

This sorts the data by the **Age** column in descending order.

3. Sorting by Multiple Columns:

```
SELECT Name, Department, Salary
FROM Employees
ORDER BY Department ASC, Salary DESC;
```

This first sorts the data by **Department** in ascending order and then by **Salary** in descending order within each department.

4. Sorting with Aliases:

```
SELECT Name, (Salary * 12) AS AnnualSalary
FROM Employees
ORDER BY AnnualSalary DESC;
```

This sorts the results based on the calculated column `AnnualSalary`.

Advantages of Sorting:

1. Data Organization:

Makes the data more readable and easier to analyze.

2. Customizable Order:

Allows sorting based on business needs.

3. Useful in Reporting:

Enhances the presentation of data in reports.

4. Efficient Retrieval:

Sorting helps in identifying specific patterns, like top performers or oldest records.

Summary:

Definition:

Sorting arranges data in ascending or descending order.

ORDER BY Clause:

Used for sorting, with options for single or multiple columns.

Examples:

Sorting by columns like age, salary, or calculated values.

Usage:

Essential for organizing and analyzing data effectively in SQL queries.

4.2 Order by Clause, Group by Clause:

These are two essential clauses in SQL used for sorting and grouping data in a meaningful way.

1. ORDER BY Clause:

The **ORDER BY** clause is used to sort the rows in the result set of a query, either in ascending or descending order.

Syntax:

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column_name [ASC|DESC];
```

ASC: Sorts in ascending order (default).
DESC: Sorts in descending order.

Examples:

a. Sorting in Ascending Order:

```
SELECT Name, Age
FROM Students
ORDER BY Age;
```

This sorts students by **Age** in ascending order.

b. Sorting in Descending Order:

```
SELECT Name, Marks
FROM Students
ORDER BY Marks DESC;
```

This sorts students by **Marks** in descending order.

c. Sorting by Multiple Columns:

```
SELECT Name, Department, Salary
FROM Employees
ORDER BY Department ASC, Salary DESC;
```

This sorts employees first by **Department** in ascending order, and then by **Salary** in descending order within each department.

2. GROUP BY Clause:

The **GROUP BY** clause is used to group rows that have the same values in specified columns. It is often used with aggregate functions like `SUM()`, `COUNT()`, `AVG()`, `MAX()`, or `MIN()` to perform

calculations on groups of data.

Syntax:

```
SELECT column1, aggregate_function(column2)
FROM table_name
GROUP BY column1;
```

Examples:

a. Grouping Data:

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department;
```

This counts the number of employees in each **Department**.

b. Using Aggregate Functions:

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY Department;
```

This calculates the average salary for each **Department**.

c. Combining GROUP BY and HAVING:

```
SELECT Department, SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department
HAVING SUM(Salary) > 50000;
```

This filters departments where the total salary exceeds **50,000**.

Differences Between ORDER BY and GROUP BY:

Feature	ORDER BY	GROUP BY
Purpose	Sorts rows in a specific order	Groups rows with the same values
Aggregate Use	Not required	Often used with aggregate functions
Clause Order	Comes after GROUP BY (if both are used)	Comes before ORDER BY
Example Use	Sorts data for presentation	Groups data for calculations or summaries

Summary:

Clause	Purpose	Example
ORDER BY	Sorts rows in ascending/descending order	`ORDER BY Age DESC;`
GROUP BY	Groups rows with the same values	`GROUP BY Department;`

Use **ORDER BY** to organize data for better readability.

Use **GROUP BY** to summarize data based on shared column values. Both are essential for effective data analysis in SQL.

4.3 Join, Types of Join: Equi Joins, Non Equi Join, Outer Join: Left, Right, Full, Self Join, Cross Join, Joining three tables:

Joins in SQL:

A **join** in SQL is used to combine rows from two or more tables based on a related column. Joins help retrieve data spread across multiple tables by establishing relationships between them.

Types of Joins:

1. Equi Join:

Combines rows from two tables based on an equality condition.

Uses the `=` operator in the `ON` clause.

Example:

```
SELECT Students.StudentID, Students.Name, Grades.Grade
FROM Students
JOIN Grades
ON Students.StudentID = Grades.StudentID;
```

This matches rows where `StudentID` is the same in both tables.

2. Non-Equi Join:

Combines rows based on a condition that is not equality (e.g., `<, >, <=, >=`).

Example:

```
SELECT Employees.Name, Salaries.Salary
FROM Employees
JOIN Salaries
ON Employees.Experience > Salaries.MinExperience;
```

This joins rows where the employee's experience exceeds the minimum experience for a salary bracket.

3. Outer Joins:

Outer joins retrieve matched rows and also include unmatched rows from one or both tables.

a. Left Outer Join:

Returns all rows from the **left** table and matching rows from the right table. Unmatched rows from the left table contain `NULL`.

Example:

```
SELECT Students.Name, Grades.Grade
FROM Students
```

```
LEFT JOIN Grades
ON Students.StudentID = Grades.StudentID;
```

This shows all students, even if they don't have grades.

b. Right Outer Join:

Returns all rows from the **right** table and matching rows from the left table. Unmatched rows from the right table contain `NULL`.

Example:

```
SELECT Students.Name, Grades.Grade
FROM Students
RIGHT JOIN Grades
ON Students.StudentID = Grades.StudentID;
```

This shows all grades, even if no students are associated.

c. Full Outer Join:

Returns all rows from both tables, with `NULL` in columns where there is no match.

Example:

```
SELECT Students.Name, Grades.Grade
FROM Students
FULL OUTER JOIN Grades
ON Students.StudentID = Grades.StudentID;
```

This retrieves all students and all grades, whether or not they are related.

4. Self Join:

A table is joined with itself. Often used when comparing rows within the same table.

Example:

```
SELECT A.EmployeeID, A.Name, B.Name AS Manager
FROM Employees A
JOIN Employees B
ON A.ManagerID = B.EmployeeID;
```

This pairs employees with their managers.

5. Cross Join:

Combines all rows from two tables, resulting in a Cartesian product. No condition is used.

Example:

```
SELECT A.Name, B.Subject
FROM Students A
CROSS JOIN Subjects B;
```

This pairs every student with every subject.

6. Joining Three Tables:

Tables can be joined in sequence using multiple `JOIN` clauses.

Example:

```
SELECT S.Name, G.Grade, C.CourseName
FROM Students S
JOIN Grades G
ON S.StudentID = G.StudentID
JOIN Courses C
ON G.CourseID = C.CourseID;
```

This retrieves student names, grades, and course names.

Summary of Joins:

Type	Description	Example
Equi Join	Matches rows using the `=` operator.	`JOIN … ON A.ID = B.ID`
Non-Equi Join	Matches rows using other operators like `<, >, <=, >=`.	`ON A.Salary > B.MinSalary`
Left Outer Join	Includes all rows from the left table.	`LEFT JOIN`
Right Outer Join	Includes all rows from the right table.	`RIGHT JOIN`
Full Outer Join	Includes all rows from both tables.	`FULL OUTER JOIN`
Self Join	Joins a table to itself.	`JOIN Employees A ON …`
Cross Join	Returns the Cartesian product of two tables.	`CROSS JOIN`

Joins allow seamless data retrieval across related tables, making SQL a powerful tool for managing relational databases.

4.4 Subqueries & its types:

Subqueries in SQL:

A **subquery** is a query nested inside another query. It is used to perform intermediate calculations or retrieve data required for the main query. Subqueries can appear in `SELECT`, `INSERT`, `UPDATE`, or `DELETE` statements.

Key Features of Subqueries:

1. A subquery is enclosed in parentheses `()` and executes before the main query.
2. It can return a single value, a list of values, or a table.
3. Subqueries can be nested multiple levels deep, though performance might degrade for complex queries.

Types of Subqueries:

1. Single-Row Subquery:

Returns **only one row** as output.

Often used with operators like `=`, `<`, `>`, `<=`, `>=`.

Example:

```
SELECT Name, Salary
FROM Employees
WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```

This retrieves employees earning more than the average salary.

2. Multi-Row Subquery:

Returns **multiple rows** as output.
Used with operators like `IN`, `ANY`, `ALL`.

Example:

```
SELECT Name
FROM Students
WHERE StudentID IN (SELECT StudentID FROM Grades WHERE Grade = 'A');
```

This retrieves names of students with an "A" grade.

3. Correlated Subquery:

Depends on the outer query for its execution. Each row from the outer query is evaluated with the subquery.
Typically slower than non-correlated subqueries.

Example:

```
SELECT E1.Name, E1.Salary
FROM Employees E1
WHERE E1.Salary > (SELECT AVG(E2.Salary) FROM Employees E2 WHERE E1.Department =
E2.Department);
```

This retrieves employees earning more than the average salary in their department.

4. Nested Subquery:

A subquery within another subquery.
Can involve multiple levels of nesting.

Example:

```
SELECT Name
FROM Students
WHERE StudentID = (SELECT TOP_StudentID
        FROM (SELECT StudentID, MAX(Grade) AS TOP_Grade FROM Grades));
```

This retrieves the name of the student with the highest grade.

5. Scalar Subquery:

Returns a **single value** (scalar) as output.

Can be used in place of a column or constant.

Example:

```
SELECT Name, (SELECT AVG(Salary) FROM Employees) AS AvgSalary
FROM Employees;
```

This adds the average salary as a column for all employees.

Subquery Placement:

Subqueries can be used in:

1. SELECT Clause:

```
SELECT Name, (SELECT AVG(Salary) FROM Employees) AS AvgSalary
FROM Employees;
```

2. FROM Clause:

```
SELECT Department, MAX(Salary)
FROM (SELECT * FROM Employees WHERE Age > 30) AS SubTable
GROUP BY Department;
```

3. WHERE Clause:

```
SELECT Name
FROM Employees
WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```

Advantages of Subqueries:

1. Simplifies complex queries by breaking them into smaller parts.
2. Enables dynamic calculations and filtering.
3. Increases query modularity and readability.

Summary of Subqueries:

Type	Description	Operators Used
Single-Row	Returns one row.	`=`, `<`, `>`
Multi-Row	Returns multiple rows	`IN`, `ANY`, `ALL`
Correlated	Dependent on the outer query.	Depends on context
Nested	A subquery inside another subquery.	Any
Scalar	Returns a single value for use as a column or constant.	Any

Subqueries are powerful tools for making SQL queries dynamic, modular, and flexible for handling complex database operations.

4.5 Overview of PL / SQL:

PL/SQL Overview:

PL/SQL (Procedural Language/Structured Query Language) is an extension of SQL used in Oracle databases. It combines SQL's data manipulation capabilities with procedural programming features like loops, conditions, and exception handling. PL/SQL enables developers to write code that interacts with the database more dynamically and efficiently.

4.5.1 Declarations Section:

The **Declarations Section** in a PL/SQL block is where variables, constants, cursors, and user-defined types are declared. This section is optional and precedes the executable statements in the PL/SQL block.

Structure of a PL/SQL Block:

A PL/SQL block has the following structure:

```
DECLARE
    Declarations Section
    variable_name datatype [NOT NULL] [:= value];
    constant_name CONSTANT datatype := value;

BEGIN
    Executable Section
    (PL/SQL code and SQL statements go here)

EXCEPTION
    Exception Handling Section
    (Handles runtime errors)
END;
```

DECLARE:

Marks the declarations section.

BEGIN:

Marks the start of the executable section.

EXCEPTION:

Optional; handles errors.

END:

Ends the block.

Components of the Declarations Section:

1. Variables:

Variables store temporary values during the execution of the PL/SQL block.

Syntax:

```
variable_name datatype [NOT NULL] [:= value];
```

Example:

```
DECLARE
    emp_name VARCHAR2(50); -- Declares a variable
    emp_id NUMBER := 101; -- Declares and initializes a variable
```

2. Constants:

Constants store values that do not change during execution.

Syntax:

```
constant_name CONSTANT datatype := value;
```

Example:

```
DECLARE
    pi CONSTANT NUMBER := 3.14; -- Declares a constant
```

3. Cursors:

Cursors are used to handle query result sets in PL/SQL.

Syntax:

```
CURSOR cursor_name IS select_statement;
```

Example:

```
DECLARE
    CURSOR emp_cursor IS SELECT emp_id, emp_name FROM employees;
```

4. User-Defined Types:

Used to declare custom types like records or tables.

Example:

```
DECLARE
    TYPE emp_record IS RECORD (
        emp_id NUMBER,
        emp_name VARCHAR2(50)
    );
    employee emp_record; -- Declares a variable of custom type
```

Importance of Declarations Section:

1. Initialization:

Declares and initializes variables or constants.

2. Modularity:

Centralizes variable definitions, making code easier to maintain.

3. Error Handling:

Prepares variables for use in exceptions or conditions.

4. Performance:

Improves performance by reducing the number of SQL calls.

Example PL/SQL Block with Declarations:

```
DECLARE
    emp_id NUMBER := 101; -- Variable
    emp_name VARCHAR2(50); -- Variable
    pi CONSTANT NUMBER := 3.14; -- Constant

BEGIN
    Assign a value to the variable
    emp_name := 'John Doe';

    Display the values
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_id);
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || emp_name);
    DBMS_OUTPUT.PUT_LINE('PI Value: ' || pi);
END;
```

Output:

Employee ID: 101
Employee Name: John Doe
PI Value: 3.14

Summary:

Component	Description	Example
Variables	Stores temporary values.	`emp_id NUMBER := 101;`
Constants	Stores value that do not change.	`pi CONSTANT NUMBER := 3.14;`
Cursors	Handles query result sets.	`CURSOR emp_cursor IS SELECT ...;`
User-Defined Types	Declares custom data types like records or tables.	`TYPE emp_record IS RECORD (...);`

The declarations section is fundamental for setting up variables and structures used in the executable part of a PL/SQL program. It ensures clean and efficient database interaction.

4.5.2 Executable Commands Section:

The **Executable Commands Section** in PL/SQL is where the actual logic of the program is implemented. It contains statements that perform operations such as data manipulation, variable assignments, control structures (loops, conditions), and SQL queries.

This section is **mandatory** in every PL/SQL block and begins with the keyword `BEGIN`.

Structure of the Executable Section:

The executable section is part of the overall PL/SQL block structure:

```
DECLARE
    Declarations Section (optional)
    variable_name datatype [:= value];
```

```
BEGIN
        Executable Commands Section (mandatory)
        SQL statements, PL/SQL logic, and control structures go here

EXCEPTION
        Exception Handling Section (optional)
        Code to handle errors

END;
```

Key Components of the Executable Section:

1. Variable Assignments:

Variables can be assigned values using the `:=` operator.

variable_name := value;

Example:

```
BEGIN
        emp_name := 'John Doe';

END;
```

2. SQL Statements:

SQL commands like `SELECT`, `INSERT`, `UPDATE`, and `DELETE` are used within the PL/SQL block to interact with the database.

Example:

```
BEGIN
        UPDATE Employees
        SET Salary = Salary + 1000
        WHERE Department = 'IT';

END;
```

3. Control Structures:

Conditional Statements:

`IF...THEN...ELSE` logic to control flow.

Loops:

Iterative structures like `FOR`, `WHILE`, or `LOOP`.

Example:

```
BEGIN
        IF salary > 50000 THEN
                DBMS_OUTPUT.PUT_LINE('High Salary');
```

```
        ELSE
            DBMS_OUTPUT.PUT_LINE('Normal Salary');

        END IF;

    END;
```

4. Exception Handling:

Errors can be caught and handled in the `EXCEPTION` section if they occur during execution.

Example:

```
    BEGIN
        SELECT Salary INTO emp_salary FROM Employees WHERE EmpID = 101;

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No record found.');

    END;
```

5. Built-In Procedures:

Commonly used procedures like `DBMS_OUTPUT.PUT_LINE` for debugging or displaying output.

Example of an Executable Section:

```
DECLARE
    emp_name VARCHAR2(50);
    emp_salary NUMBER;

BEGIN
    Assigning a value to a variable
    emp_name := 'Alice';

    SQL Query to fetch data
    SELECT Salary INTO emp_salary FROM Employees WHERE Name = emp_name;

    Conditional logic
    IF emp_salary > 50000 THEN
        DBMS_OUTPUT.PUT_LINE(emp_name || ' has a high salary.');

    ELSE
        DBMS_OUTPUT.PUT_LINE(emp_name || ' has a normal salary.');

    END IF;

    Update operation
    UPDATE Employees
```

```
        SET Bonus = Bonus + 500
        WHERE Name = emp_name;

    END;
```

Characteristics of the Executable Section:

1. **Mandatory:**
   Every PL/SQL block must have this section.

2. **Handles Logic:**
   Implements the main logic of the program.

3. **Performs SQL Operations:**
   Integrates SQL statements seamlessly.

4. **Supports Procedural Features:**
   Includes loops, conditions, and function calls.

Summary:

Feature	Description	Example
Variable Assignments	Assigns values to variables.	`emp_name := 'John Doe';`
SQL Operations	Executes `SELECT`, `INSERT`, `UPDATE`, `DELETE`.	`SELECT Salary INTO emp_salary...;`
Control Structures	Implements conditional and iterative logic.	`IF...THEN`, `FOR`, `WHILE`
Error Handling	Captures runtime errors using `EXCEPTION`.	`WHEN NO_DATA_FOUND THEN...`
Output Statements	Debugs or displays results.	`DBMS_OUTPUT.PUT_LINE('Message');`

The **Executable Commands Section** is the core of a PL/SQL block where the program's operations are carried out, making it crucial for implementing logic and interacting with the database.

4.5.3 Exception Handling Section:

The **Exception Handling Section** in PL/SQL is used to manage runtime errors or exceptions that occur during the execution of a PL/SQL block. This section ensures that the program does not terminate abruptly and provides a way to handle errors gracefully.

Structure of the Exception Handling Section:

The Exception Handling Section is optional and is defined using the `EXCEPTION` keyword. It is placed after the `BEGIN` section and before the `END` keyword in a PL/SQL block.

```
    DECLARE
        Declarations Section (optional)

    BEGIN
        Executable Commands Section
        Code that might raise exceptions
```

```
EXCEPTION
        Exception Handling Section
        WHEN exception_name THEN

        Actions to handle the exception
        WHEN OTHERS THEN

        Default actions for all other exceptions

END;
```

Types of Exceptions:

1. Predefined Exceptions:

These are built-in exceptions provided by Oracle. Examples include:

`NO_DATA_FOUND`:
Raised when a `SELECT INTO` statement returns no rows.

`TOO_MANY_ROWS`:
Raised when a `SELECT INTO` statement returns multiple rows.

`ZERO_DIVIDE`:
Raised when dividing a number by zero.

`INVALID_CURSOR`:
Raised when an invalid cursor operation is attempted.

2. User-Defined Exceptions:

These are explicitly defined by the user using the `EXCEPTION` keyword in the declarations section.

Syntax of Exception Handling:

Handling Predefined Exceptions:

```
BEGIN
        Code that might raise an exception
        SELECT Salary INTO emp_salary FROM Employees WHERE EmpID = 101;

EXCEPTION
        WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No record found.');
        WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Multiple records found.');

END;
```

Defining and Handling User-Defined Exceptions:

```
DECLARE
```

```
        emp_salary NUMBER;
        salary_exception EXCEPTION; -- User-defined exception

    BEGIN
        SELECT Salary INTO emp_salary FROM Employees WHERE EmpID = 101;

        IF emp_salary < 0 THEN
            RAISE salary_exception; -- Raising the exception

        END IF;

    EXCEPTION
        WHEN salary_exception THEN
        DBMS_OUTPUT.PUT_LINE('Salary cannot be negative.');

    END;
```

Handling All Exceptions:

The `WHEN OTHERS` clause is used to catch any exception not explicitly handled.

```
    BEGIN
        SELECT Salary INTO emp_salary FROM Employees WHERE EmpID = 101;

    EXCEPTION
        WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred.');

    END;
```

Advantages of Exception Handling:

1. **Error Management:**
   Prevents abrupt termination by catching runtime errors.

2. **Debugging:**
   Provides detailed error messages for easier debugging.

3. **Program Continuity:**
   Allows the program to proceed or perform alternative actions despite errors.

4. **Custom Error Logic:**
   Enables defining custom error-handling logic for specific cases.

Example: Complete PL/SQL Block with Exception Handling:

```
DECLARE
    emp_salary NUMBER;
    salary_exception EXCEPTION;
```

```
BEGIN
    Attempt to fetch data
    SELECT Salary INTO emp_salary FROM Employees WHERE EmpID = 101;

    Custom logic to raise an exception
    IF emp_salary < 0 THEN
        RAISE salary_exception;

    END IF;

    Display fetched salary
    DBMS_OUTPUT.PUT_LINE('Employee Salary: ' || emp_salary);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No employee found with the given ID.');
    WHEN salary_exception THEN
    DBMS_OUTPUT.PUT_LINE('Error: Salary cannot be negative.');
    WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An unexpected error occurred.');

END;
```

Summary:

Type of Exception	Description	Example
Predefined Exception	Built-in Oracle exceptions like `NO_DATA_FOUND`, `ZERO_DIVIDE`.	`WHEN NO_DATA_FOUND THEN…`
User-Defined Exception	Custom exceptions declared using the `EXCEPTION` keyword.	`RAISE exception_name;`
WHEN OTHERS	Catches all exceptions not explicitly handled.	`WHEN OTHERS THEN…`

The Exception Handling Section is critical for building robust and error-resilient PL/SQL programs, ensuring smooth execution even in the face of unexpected issues.